

Beyond Static RAG—Building Autonomous Agents

From Retrieval to Agency

- The Evolution:** We started with RAG to solve context limitations. Now, we move to **Agents**—systems that can interact with the world.

- The Goal:** To move from models that just *read* to models that *act*—automating tasks like data analysis, coding, and negotiation.

- Core Definition:** An agent is anything that can perceive its environment and act upon it.

- The Stack:**

- 1.**Environment:** The domain (e.g., the internet, a file system, a game) .

- 2.**Tools:** The actions the agent can take (e.g., search, SQL execution).

- 3.**Planner:** The AI brain that orchestrates the actions.

Speaker Notes: "Welcome back. In the first half of Chapter 6, we discussed RAG as a way to give models memory. But RAG is just one piece of the puzzle. Today, we explore **Agents**. If RAG is a scholar reading a book, an Agent is an intern with a laptop and a credit card. It doesn't just know things; it *does* things. We will explore how to build these systems, how they plan, and crucially, where they fail."

RAG for Structured Data (The SQL Agent)

Visual: A diagram showing User Query -> Text-to-SQL -> Database -> Response.

- **The Challenge:** Most enterprise data isn't text; it's **tabular** (SQL tables, Excel sheets).

- **Case Study: Kitty Vogue:**

- *Query:* "How many units of Fruity Fedora were sold in the last 7 days?".

- *Problem:* Standard semantic retrieval fails on aggregations (SUM, COUNT).

- **The Solution: Text-to-SQL Workflow :**

- 1. **Text-to-SQL:** Model converts user query + schema into SQL.

- 2. **Execution:** Run the SQL query against the database.

- 3. **Generation:** Use the SQL result to answer the user.

- **Key Insight:** This is a primitive agent. It uses a tool (SQL Executor) to answer a question it couldn't solve with internal knowledge alone.

Speaker Notes:

"Before we get to full autonomy, let's look at a specialized agent: the SQL agent. If I ask, 'What were my sales last week?', a vector database is useless. We need to compute. We treat the database as a tool. The model generates SQL, executes it, and reads the result. This is the bridge between unstructured RAG and structured data analysis."

The Agentic Framework

Visual: A loop diagram: Perception -> Planning -> Action -> Observation -> Reflection.

- **Environment & Actions:** An agent is defined by its environment and the actions it can perform.
 - *Example:* **SWE-agent** (Software Engineering Agent).
 - *Environment:* Computer terminal and file system.
 - *Actions:* Maps_repo, search_files, edit_lines.
- **High Stakes:** Agents face "compound mistakes." If an agent needs 10 steps and has 95% accuracy per step, the final success rate drops to ~60%.
- **Autonomy:** The goal is to save human time by allowing the model to perform long sequences of actions independently.

Speaker Notes:

"What makes an agent? It's the environment. A chess bot lives on a board; a coding agent like SWE-agent lives in your terminal. The biggest challenge here is *compound error*. If your agent tries to fix a bug and takes 10 steps, a single slip-up ruins the whole task. This is why we need robust planning."

The Tool Inventory

Visual: Icons representing different tool categories (Search, Calculator, API).

- **Knowledge Augmentation:** Tools that give the agent context.
 - Retrievers (RAG), Google Search, Internal Wikis.
 - Prevents the model from going "stale" (outdated training data).
- **Capability Extension:** Fixing inherent model weaknesses.
 - *Math:* Models are bad at arithmetic; give them a Calculator tool .
 - *Code:* Use a Code Interpreter to run Python for data analysis.
- **Write Actions:** Changing the environment.
 - Sending emails, booking tickets, modifying databases .
 - **Security Risk:** "Prompt Injection" can now lead to real-world damage (e.g., bank transfers).

Speaker Notes: "We give agents 'tools' to compensate for their weaknesses. Don't train an LLM to do math; just give it a calculator. The dangerous part is 'Write Actions'. Reading a database is safe; updating it is risky. As Chip Huyen notes, you wouldn't give an intern root access to your production DB—treat your agent with the same caution."

Planning & Reasoning

Visual: A flowchart showing Task Decomposition.

- **The Planner:** The component that breaks a complex task into a sequence of manageable actions.
- **Control Flows:** Plans aren't just lists; they are programs .
 - **Sequential:** Step A -> Step B (e.g., Translate SQL -> Execute).
 - **Parallel:** Task A & B simultaneously (e.g., browse 10 websites at once).
 - **If/Else:** Decision making (e.g., If stock earnings > X, Buy; else Sell).
 - **Loops:** Repeat until condition met (e.g., debug until tests pass).
- **Decoupling:** To avoid wasting money, generate the plan *first*, validate it, and *then* execute it.

Speaker Notes:

"Planning is the brain of the agent. It's not enough to just say 'do this'. The agent must understand dependencies. Can I run these searches in parallel? Do I need to loop this action until I get a valid result? Advanced agents don't just act; they write programs to solve your problem."

Reflection & Error Correction

Visual: The ReAct pattern diagram (Thought -> Act -> Observation).

- **ReAct Pattern:** Interleaving reasoning (planning) and acting.

- *Thought:* "I need to find the author."
- *Act:* search(book_title)
- *Observation:* "Author is Liu Cixin."
- *Thought:* "Now I need his other books."

- **Reflexion:** An advanced framework where the agent explicitly critiques its own past attempts.

- Separates "Actor" from "Evaluator".
- If a plan fails, the agent reflects on *why* and proposes a new "trajectory".

- **Self-Correction:** Essential for coding agents (e.g., seeing a unit test fail and rewriting the code) .

Speaker Notes:

"An agent that runs blindly is a bad agent. The ReAct pattern—Reason, Act, Observe—is the standard for a reason. It allows the agent to adjust. If it searches for 'Apple' and gets fruit instead of tech, it sees the observation and corrects its next search. This loop is what makes agents robust."

Failure Modes & Evaluation

Visual: A dashboard showing "Planning Failures" and "Tool Failures".

- Planning Failures:**

- Invalid Tool:** Trying to call Google Search when it doesn't exist.
- Hallucinated Parameters:** Calling lbs_to_kg(100, "pounds") when the function only takes one argument.
- Looping:** Getting stuck in a loop without achieving the goal.

- Tool Failures:** The tool is called correctly, but returns wrong data (e.g., SQL query is syntactically correct but logically wrong) .

- Evaluation Strategy:**

- Measure **Efficiency:** Steps per task, Cost per task.
- Measure **Validity:** % of valid tool calls, % of successful plans .

Speaker Notes: "How do we know if an agent is good? We look at failures. Does it try to invent tools that don't exist? Does it get stuck in infinite loops? We need to evaluate not just the final answer, but the *process*. Did it take 100 steps to do a 5-step task? That's an efficiency failure."

Memory Systems

Visual: Architecture diagram showing Internal, Short-term, and Long-term memory.

- Hierarchy of Memory:**

- 1.**Internal Knowledge:** The model's training weights (Static).
- 2.**Short-Term Memory:** The Context Window (Limited, fast access).
- 3.**Long-Term Memory:** External databases/RAG (Unlimited, retrieval needed).

- Memory Management:**

- Overflow:** Moving data from Context to Long-term storage when the window fills up.
- FIFO vs. Summarization:** First-In-First-Out is simple but risky (you might lose the original instruction) . Summarization is better for reducing redundancy.

Speaker Notes: "Finally, agents need memory. If you have a long conversation, the context window fills up. You can't just delete the old stuff—you might delete the user's instructions! We need systems that move short-term context into long-term storage, effectively giving the agent a permanent brain."

Conclusion

Visual: A roadmap graphic pointing towards "Autonomous Systems".

- Summary:**

- RAG** augments knowledge.

- Agents** augment capabilities through tools and planning.

- The Trade-off:** More tools = More capabilities = Higher risk of failure and security threats.

- Future:** FM (Foundation Model) Agents and RL (Reinforcement Learning) Agents will likely merge.

Speaker Notes: "We are moving from chatbots that talk to agents that work. The potential is enormous, but so are the risks. As we build these systems, we must balance capability with safety. The future belongs to agents that can plan, reflect, and remember."